

AutoGrade — Hệ thống chấm điểm code tự động

Software Architecture Document

Đồ án môn Công nghệ phần mềm cho AI — PA4
Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Table of Contents

1. Introduction.....	4
2. Architectural Goals and Constraints.....	4
3. Use-Case Model.....	5
4. Logical View.....	6
4.1 Component: Frontend (React SPA)6	
4.2 Component: Auth7	
4.3 Component: Course & Assignment8	
4.4 Component: Submission8	
4.5 Component: Grading Engine9	
4.6 Component: Database — Supabase (PostgreSQL)10	
5. Deployment.....	10
6. Implementation View	11

1. Introduction

Mục đích: Tài liệu này mô tả kiến trúc phần mềm (software architecture) của hệ thống AutoGrade — hệ thống chấm điểm code tự động phục vụ giảng dạy — theo mẫu Software Architecture Document (SAD) rút gọn của RUP. Tài liệu giúp người đọc (giảng viên hướng dẫn, thành viên nhóm) hiểu các thành phần chính của hệ thống, cách chúng giao tiếp với nhau và cách hệ thống được triển khai.

Phạm vi: Tài liệu bao quát toàn bộ hệ thống AutoGrade gồm Frontend (React SPA), Backend (Node.js/Express) và lõi chấm điểm 5 mô-đun (Grading Engine), cùng cách các thành phần này giao tiếp với Cơ sở dữ liệu (Supabase/PostgreSQL) và hai dịch vụ AI ngoài hệ thống (Gemini API, Groq API). Tài liệu không bao gồm quy trình CI/CD hay công cụ nội bộ nhóm dùng trong quá trình phát triển.

Định nghĩa, từ viết tắt:

- RUP: Rational Unified Process.
- SAD: Software Architecture Document.
- SPA: Single Page Application.
- JWT: JSON Web Token — cơ chế xác thực bằng token ký số.
- LLM: Large Language Model (mô hình ngôn ngữ lớn).
- Rubric: bộ tiêu chí và trọng số dùng để tính điểm tự động.
- Winnowing: kỹ thuật băm k-gram để phát hiện đạo văn/gian lận code (kiểu MOSS).
- PA3 / PA4: Programming Assignment 3 / 4 — hai đợt nộp đề án của môn học.

Tài liệu tham khảo:

- AutoGrade_UseCase_Specification.docx — đặc tả 14 use case của hệ thống, phiên bản 1.0.
- README.md, backend/README.md, HUONG_DAN_CHAM_DIEM.md — tài liệu kỹ thuật trong repo, mô tả cách chạy hệ thống và chi tiết 5 mô-đun chấm điểm.

Tổng quan tài liệu: Phần 2 nêu các mục tiêu và ràng buộc kiến trúc. Phần 3 nhắc lại sơ đồ use-case đã có trong tài liệu đặc tả use-case. Phần 4 (Logical View) trình bày các thành phần phần mềm và quan hệ giữa chúng. Phần 5 mô tả cách hệ thống được triển khai lên các máy/dịch vụ thực tế. Phần 6 trình bày cấu trúc thư mục mã nguồn tương ứng với các thành phần ở Phần 4.

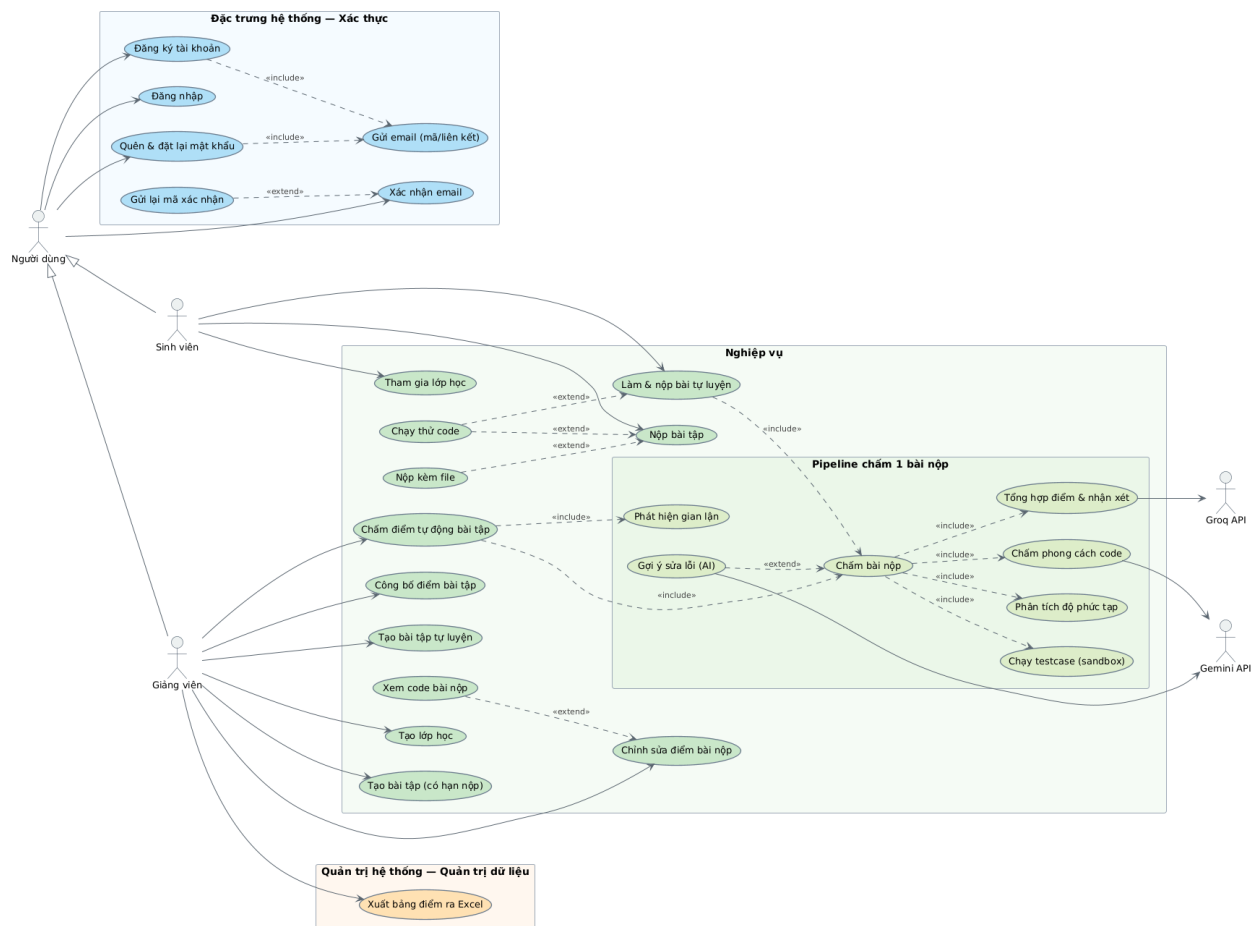
2. Architectural Goals and Constraints

- Toàn bộ hệ thống dùng TypeScript (Frontend & Backend) để chia sẻ tư duy về kiểu dữ liệu và giảm lỗi runtime.
- Kiến trúc tách rời: Frontend (SPA) và Backend (REST API) là 2 project độc lập, giao tiếp qua HTTPS + JSON, xác thực bằng JWT.
- Nền tảng: ứng dụng Web (không có bản mobile riêng); Backend yêu cầu Node.js ≥ 18 ; sandbox chấm code chạy tiến trình trực tiếp trên hệ điều hành máy chủ, không phụ thuộc Docker/WSL.
- Cơ sở dữ liệu dùng Supabase (PostgreSQL as a Service), truy cập qua API key (service_role) từ Backend — không tự vận hành hạ tầng DB, không cần bật Row-Level-Security vì Backend là điểm truy cập duy nhất.
- Bảo mật: mật khẩu mã hoá bằng bcrypt; xác thực bằng JWT có thời hạn; khoá service_role chỉ tồn tại ở Backend, không bao giờ lộ ra Frontend; xác nhận email và đặt lại mật khẩu qua mã dùng một lần có thời hạn.
- Giới hạn tài nguyên khi chấm code: mỗi testcase chạy tối đa 5 giây, biên dịch tối đa 15 giây, RAM tối đa 256MB mỗi tiến trình — để code sinh viên (không tin cậy) không thể làm treo hoặc chiếm dụng máy chủ.

- Khả năng thay thế nhà cung cấp AI: 2 mô-đun dùng LLM (Style → Gemini, Tổng kết → Groq) chỉ cần đổi biến môi trường để đổi model/nhà cung cấp, không cần sửa code; nếu thiếu API key, hệ thống tự dùng nhận xét dự phòng và điểm số vẫn đúng vì điểm được tính tất định, không phụ thuộc LLM.
- Khả năng tái lập kết quả chấm: mô-đun Độ phức tạp dùng phân tích tĩnh cấu trúc code (không đo thời gian thực nghiệm) để đảm bảo chấm lại cùng một bài luôn ra cùng một điểm.
- Ràng buộc dự án: đây là đồ án môn học, phát triển bởi một nhóm nhỏ trong một học kỳ; ưu tiên triển khai đơn giản (Supabase free tier, không cần thuê server riêng) hơn là khả năng chịu tải lớn.

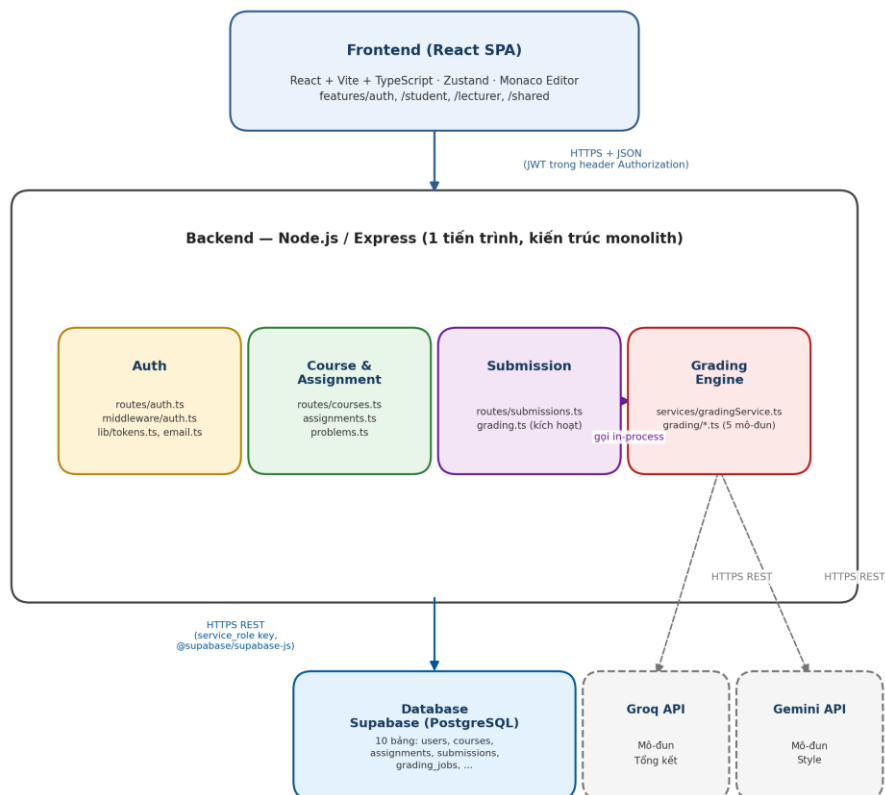
3. Use-Case Model

Sơ đồ use-case và đặc tả chi tiết của từng use case (Basic Flow, Alternative Flows, Pre-/Post-conditions) đã được trình bày đầy đủ trong tài liệu `AutoGrade_UseCase_Specification.docx` (v1.0). Hình dưới đây nhắc lại sơ đồ use-case tổng quan — gồm 2 actor chính (Sinh viên, Giảng viên) và 2 actor phụ là dịch vụ AI ngoài hệ thống (Gemini API, Groq API) — để tiện tham chiếu khi đọc phần Logical View bên dưới, vì các use case này chính là các luồng nghiệp vụ mà các component ở Phần 4 cùng nhau hiện thực.



4. Logical View

Hệ thống được chia thành 6 component chính: 1 component Frontend và 4 component phía Backend (Auth, Course & Assignment, Submission, Grading Engine) cùng chạy trong một tiến trình Node.js duy nhất — kiến trúc monolith, không tách microservice — cộng với Database. Hình dưới đây trình bày các component và quan hệ giao tiếp giữa chúng, kèm giao thức/cơ chế giao tiếp tương ứng.



Chi tiết trách nhiệm, module/file chính, kiểu dữ liệu chính và quan hệ phụ thuộc của từng component được trình bày ở các mục con 4.1–4.6. Vì Backend không dùng nhiều class theo hướng OOP (chủ yếu là hàm/route Express), “kiểu dữ liệu chính” của mỗi component được thể hiện bằng các TypeScript interface/type thay cho class.

4.1 Component: Frontend (React SPA)

Frontend là giao diện web duy nhất của hệ thống, phục vụ cả hai vai trò Sinh viên và Giảng viên (phân biệt qua route và trạng thái đăng nhập). Toàn bộ trạng thái phía client được quản lý bằng Zustand; soạn code dùng Monaco Editor; việc xuất bảng điểm ra Excel được thực hiện hoàn toàn phía client bằng thư viện xlsx, không tốn tài nguyên Backend.

Frontend (React SPA)	
Trách nhiệm	Giao diện cho Sinh viên & Giảng viên; soạn/nộp code (Monaco Editor); quản lý state (Zustand); xuất bảng điểm ra Excel phía client.
Module / file chính	- features/auth, /student, /lecturer, /shared (trang & dialog theo vai trò) - store/auth.ts, content.ts, submissions.ts (Zustand store) - lib/api.ts (HTTP client gọi Backend)
Kiểu dữ liệu chính	User, Course, Assignment, Problem, Submission, ProblemSubmission, Rubric (types/index.ts)
Giao tiếp / phụ thuộc	Gọi Backend API qua HTTPS + JSON, đính JWT ở header Authorization.

4.2 Component: Auth

Auth là component xuyên suốt (cross-cutting): mọi request từ Course & Assignment và Submission đều đi qua middleware requireAuth/requireRole của Auth trước khi được xử lý, nên quan hệ này là lời gọi hàm in-process, không phải một request mạng riêng và không được vẽ thành mũi tên ở sơ đồ tổng quan (Hình Logical View).

Auth	
Trách nhiệm	Đăng ký, xác nhận email, đăng nhập, quên/đặt lại mật khẩu; phát hành & kiểm tra JWT; middleware bảo vệ route theo vai trò (student/lecturer).
Module / file chính	- routes/auth.ts - middleware/auth.ts (requireAuth, requireRole) - lib/tokens.ts (signJwt, randomCode), lib/email.ts
Kiểu dữ liệu chính	- User { id, email, role, email_verified, ... } - JWT payload { sub, role }
Giao tiếp / phụ thuộc	Nhận HTTP request từ Frontend · đọc/ghi bảng users qua Database · gửi email qua SMTP (nodemailer) hoặc log console ở môi trường DEV.

4.3 Component: Course & Assignment

Component này gộp các nghiệp vụ quản lý lớp học và ra đề: tạo/tham gia lớp bằng mã lớp (join_code), tạo bài tập có hạn nộp kèm rubric và testcase, tạo bài luyện tập tự do công khai, và quản lý tài liệu môn học.

Course & Assignment	
Trách nhiệm	• CRUD lớp học & tham gia lớp bằng mã (join_code); ra đề bài tập kèm rubric/testcase; tạo bài luyện tập công khai; quản lý tài liệu môn học.
Module / file chính	• routes/courses.ts, assignments.ts, problems.ts
Kiểu dữ liệu chính	• Course, Enrollment, Assignment { rubric: RubricConfig }, Problem, Testcase, Material
Giao tiếp / phụ thuộc	• HTTP từ Frontend · qua middleware của Auth để xác thực/phân quyền · đọc/ghi Database · lưu file (PDF đề, tài liệu) qua lib/upload.ts.

4.4 Component: Submission

Component tiếp nhận bài nộp (cả bài có hạn nộp và bài luyện tập), giữ lịch sử, và là nơi kích hoạt Grading Engine (gọi hàm in-process, không qua HTTP nội bộ vì cùng một tiến trình Node.js). Sau khi có kết quả, component này cũng xử lý việc Giảng viên ghi đề điểm, công bố/hoàn tất điểm và dựng bảng điểm tổng hợp (gradebook).

Submission	
Trách nhiệm	• Nhận bài nộp (assignment & practice); theo dõi trạng thái & lịch sử; kích hoạt chấm điểm; ghi đề điểm; công bố/hoàn tất điểm; dựng bảng điểm (gradebook).
Module / file chính	• routes/submissions.ts • routes/grading.ts (kích hoạt chấm) • routes/problems.ts (phần nộp bài luyện tập)
Kiểu dữ liệu chính	• Submission, ProblemSubmission, GradingJob
Giao tiếp / phụ thuộc	• HTTP từ Frontend · gọi in-process hàm gradeAssignment()/gradeProblemSubmission() của Grading Engine · đọc/ghi Database.

4.5 Component: Grading Engine

Đây là component phức tạp nhất và cũng là trọng tâm kỹ thuật của đồ án: hiện thực đúng pipeline 5 mô-đun theo thứ tự Gian lận (gate) → Testcase → (Độ phức tạp || Phong cách, chạy song song) → Tổng kết, đúng như mô tả trong HUONG_DAN_CHAM_DIEM.md và use case “Chấm điểm tự động bài tập” của tài liệu đặc tả use-case.

Grading Engine	
Trách nhiệm	Thực thi pipeline chấm 5 mô-đun: Gian lận → Testcase → (Độ phức tạp Phong cách) → Tổng kết, đúng luồng rẽ nhánh trong HUONG_DAN_CHAM_DIEM.md.
Module / file chính	- services/gradingService.ts (điều phối) - grading/pipeline.ts, testcase.ts, sandbox.ts, - complexity.ts, style.ts, plagiarism.ts, aggregate.ts
Kiểu dữ liệu chính	RubricConfig, GradeInput, GradeResult, CriteriaResult, TestcaseRunResult, StyleOptions (interface, không dùng class)
Giao tiếp / phụ thuộc	Được Submission gọi in-process · gọi HTTPS REST tới Gemini API & Groq API · chạy tiến trình con biên dịch/thực thi code trong sandbox cách ly.

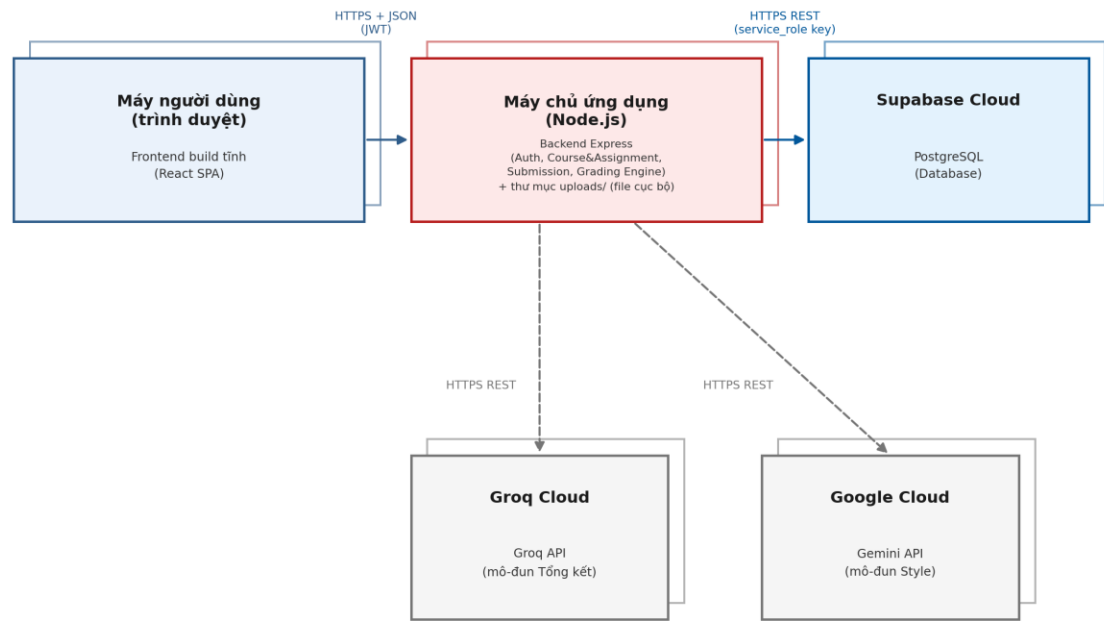
4.6 Component: Database — Supabase (PostgreSQL)

Lớp lưu trữ dưới cùng của hệ thống. Toàn bộ 10 bảng dữ liệu (users, courses, enrollments, assignments, problems, testcases, materials, submissions, problem_submissions, grading_jobs) được định nghĩa trong backend/db/schema.sql và chỉ được truy cập từ Backend.

Database — Supabase (PostgreSQL)	
Trách nhiệm	Lưu trữ toàn bộ dữ liệu bền vững của hệ thống.
Bảng chính	- users, courses, enrollments, assignments, problems, - testcases, materials, submissions, problem_submissions, grading_jobs
Truy cập	Chỉ Backend truy cập, qua @supabase/supabase-js dùng service_role API key (REST nội bộ của Supabase) — không bật Row-Level-Security vì Backend là điểm truy cập DUY NHẤT.
Phụ thuộc	Không có — là lớp lưu trữ dưới cùng.

5. Deployment

Vì đây là đồ án học phần, hệ thống hiện được triển khai ở quy mô đơn giản, ưu tiên để chạy demo hơn là khả năng chịu tải lớn. Sơ đồ dưới đây mô tả cách các component ở Phần 4 được ánh xạ vào các máy/dịch vụ thực tế.



- Máy người dùng (trình duyệt của Sinh viên/Giảng viên): chạy Frontend đã build tĩnh (npm run build) hoặc Vite dev server khi phát triển; không cần cài gì thêm ngoài một trình duyệt hiện đại.

- Máy chủ ứng dụng (Node.js ≥ 18): chạy toàn bộ Backend trong một tiến trình Express duy nhất — cả 4 component Auth, Course & Assignment, Submission, Grading Engine cùng nằm trên một máy/tiến trình; máy này cần có sẵn toolchain chấm code (python3, g++, javac tùy ngôn ngữ nhóm hỗ trợ) và là nơi lưu file upload cục bộ (thư mục uploads/ — đề PDF, tài liệu môn học, file bài nộp).
- Supabase Cloud: dịch vụ PostgreSQL-as-a-Service do Supabase vận hành; Backend kết nối qua API key (service_role), không cần tự quản lý server database.
- Groq Cloud / Google Cloud (Gemini): hai dịch vụ AI bên thứ ba mà Grading Engine gọi qua HTTPS REST khi chấm điểm (mô-đun Phong cách gọi Gemini, mô-đun Tổng kết gọi Groq); nếu mất kết nối Internet hoặc thiếu API key, hệ thống tự chuyển sang nhận xét dự phòng và điểm số vẫn được tính đúng.

Giới hạn hiện tại: hệ thống chưa dùng container hoá (Docker) hay object storage (S3/Supabase Storage) cho file upload; sandbox chấm code chạy tiến trình con trực tiếp trên máy chủ ứng dụng. Mục 8 của HUONG_DAN_CHAM_DIEM.md đã ghi rõ đây là điểm cần cải thiện (cô lập mạnh hơn bằng Docker/nsjail/firejail) nếu triển khai thật ra môi trường công khai.

6. Implementation View

Cấu trúc thư mục mã nguồn ánh xạ khá trực tiếp tới các component ở Phần 4: mỗi thư mục trong backend/src/ tương ứng với đúng một component (routes/ chia theo Auth / Course & Assignment / Submission; grading/ cùng services/ là Grading Engine); frontend/src/ được tổ chức theo tính năng/vai trò (feature-based) hơn là theo layer kỹ thuật.

```

backend/src/
├── config.ts           // đọc & validate biến môi trường (.env)
├── db.ts, supabase.ts // helper bọc Supabase client (service_role)
├── index.ts           // khởi tạo Express app, mount router
├── middleware/
│   └── auth.ts        // requireAuth, requireRole           → Auth
├── lib/
│   ├── tokens.ts, email.ts // JWT, mã xác nhận, gửi email       → Auth
│   ├── http.ts, serialize.ts // ApiError, asyncHandler, sXxx()
│   └── upload.ts        // lưu file lên thư mục uploads/
├── routes/
│   ├── auth.ts        → Auth
│   └── courses.ts, assignments.ts, problems.ts → Course & Assignment
├── submissions.ts, grading.ts → Submission
├── services/
│   └── gradingService.ts // điều phối pipeline chấm           → Grading Engine
├── grading/
│   ├── pipeline.ts, sandbox.ts, testcase.ts
│   ├── complexity.ts, style.ts, plagiarism.ts, aggregate.ts
│   └── types.ts        → Grading Engine

```

```

frontend/src/
├── App.tsx, main.tsx
├── components/
│   ├── layout/ // AppHeader, AppShell, BackButton, Logo
│   ├── auth/   // ProtectedRoute
│   └── ui/     // 13 UI primitive: button, card, dialog, input, table, tabs, ...
├── features/
│   └── auth/    // LoginPage, RegisterPage, VerifyEmailPage, Forgot/ResetPasswordPage

```

```
|   ├── student/    // StudentDashboard, CourseDetailPage, AssignmentPage,
PracticeDetailPage, ...
|   ├── lecturer/  // LecturerDashboard, LecturerCourseDetailPage, GradeAssignmentPage, ...
|   └── shared/    // CourseSections, PracticeBrowser, ProfileDialog, course-bits, problem-
bits
└── store/         // auth.ts, content.ts, submissions.ts (Zustand)
    └── lib/       // api.ts, rubric.ts, term.ts, toast.ts, utils.ts
        └── types/index.ts
```